

200 AI Engineering Interview Questions — Enhanced Edition

Every question from your original document, answered fully and correctly in one place — depth, corrections, diagrams, and references to your actual RAG code folded directly into each answer.

Source: “200 AI Engineering Interview Questions with Answers — 2025 Edition” (@sambit.ai.tech). Your excerpt contains 140 of the 200 numbered questions — the rest were never in the document you shared. All 140 are answered below, in original order and numbering, nothing dropped. Where an entry references your own code, it’s marked .

LLM FUNDAMENTALS (Q1-20)

1. What is a Large Language Model (LLM)?

A neural network trained on massive text to predict the next token — mathematically, it’s a probability distribution $P(\text{next_token} \mid \text{previous_tokens})$. It’s built on the **Transformer architecture** (see Q175 for the full diagram), using self-attention as its core mechanism so every token can relate to every other token in the input. Because of this, an LLM can perform many tasks zero-shot — no task-specific training needed — simply by being prompted correctly.

Key specs worth knowing cold: Claude supports up to 200K tokens of context; vocabulary is typically 50K-128K tokens; a 7B-parameter model needs roughly 14GB of memory just to load (fp16). Loss decreases predictably as the model scales — $\text{Loss} \propto N^{(-\alpha)}$, $\alpha \approx 0.07$ — and Chinchilla’s scaling law (Q87) says the compute-optimal ratio is about 20 training tokens per parameter.

What it’s *not*: not a database (it hallucinates rather than looks things up — Q5), not deterministic (sampling introduces randomness — Q3), and not “reasoning” in the human sense so much as extremely capable pattern completion at scale.

2. What is tokenization and why does it matter?

Tokenization converts raw text into the tokens (subwords or words) that a model actually processes — it's the first step of every LLM pipeline and it directly affects vocabulary size, sequence length, cost, and multilingual coverage. **BPE** (Byte-Pair Encoding) and **SentencePiece** are the dominant algorithms in production, both working by greedily merging frequent character/subword pairs into a fixed vocabulary.

In practice, English text runs about 1.3 characters per token, while CJK languages (Chinese, Japanese, Korean) run closer to 1 character per token — a 3-5x token (and therefore cost) expansion for the same amount of content in those languages. A subtle but important gotcha: using the wrong tokenizer for a model (e.g. a GPT tokenizer on a Llama model) produces garbage output, since token IDs mean entirely different things across vocabularies.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")
tokens = tokenizer.encode("What is RAG?")    # → ~6 tokens
```

3. What is temperature in LLM generation?

Temperature scales the logits before the softmax step: $P(\text{token}) = \text{softmax}(\text{logits} / T)$. As T approaches 0, the distribution sharpens toward the single most likely token ($T=0$ is greedy decoding, fully deterministic); as T rises toward 1 and beyond, the distribution flattens and generation becomes progressively more random and creative. $T \approx 0.7$ is a common default that balances creativity against coherence. Crucially, temperature controls *diversity of outputs*, not the model's underlying knowledge — the weights never change.

Practical guidance: use $T=0-0.1$ for deterministic tasks (SQL generation, data extraction, customer support where consistency matters), and $T=0.7-0.9$ for open-ended chat or creative writing. Don't assume the default is right for your use case — A/B test it.

4. What is the difference between zero-shot, few-shot prompting?

Zero-shot prompting describes the task in words only, with no examples — the model relies entirely on its pretraining and instruction-tuning to understand what's wanted. **Few-shot** prompting includes a handful of example input-output pairs directly in the prompt to guide the model's format and reasoning style; more examples generally improve accuracy on structured tasks like classification or extraction, though returns diminish fast

— most tasks plateau around 3-5 examples. One detail worth knowing: the *last* example in a few-shot prompt tends to have outsized influence on the output (a recency bias), so put your strongest, most representative example last rather than first.

5. What is hallucination in LLMs and why does it occur?

Hallucination is when a model generates plausible-sounding but factually incorrect content — a direct consequence of how it's trained: the model optimizes for next-token *plausibility*, not for factual accuracy, so it's fundamentally pattern-completing rather than fact-checking itself. It's caused by knowledge gaps in training data, distribution shift (queries unlike anything seen in training), and overconfident prediction (the model doesn't "know what it doesn't know").

Mitigation, ranked by real-world effectiveness: **RAG** grounds answers in retrieved documents and cuts hallucination roughly 40-60% in practice (see Q11); fine-tuning on factual data helps modestly; prompt engineering ("answer only from the provided context") has limited effect on its own; and an external verification layer — never trusting the model's output alone for anything high-stakes — is the most important production safeguard. Nothing eliminates hallucination entirely; the goal is always mitigation plus verification, not elimination.

6. What is prompt engineering?

Prompt engineering is the practice of crafting inputs to reliably elicit desired outputs from an LLM, without touching the model's weights. Core techniques include zero-shot and few-shot examples, chain-of-thought reasoning (Q21), and persona/role setting (Q27) — and it's a genuinely critical skill, since better prompts frequently beat bigger models on real tasks.

Treat it as an iterative loop rather than a one-shot write: draft a baseline prompt → test on 10-20 diverse inputs → identify failure patterns → add targeted examples or constraints for those failures → measure again → stop once gains drop below roughly 1% per iteration. One caveat worth internalizing: prompts don't transfer cleanly between models — a prompt tuned for GPT-4 will often behave differently on Claude or Llama, so re-test after any model swap or upgrade.

7. What is the context window of an LLM?

The context window is the maximum number of tokens — input plus output combined — that a model can process in a single forward pass. It's fundamentally limited by attention's $O(n^2)$ memory cost (Q13): doubling the context roughly quadruples the compute/memory needed for attention, so larger context windows are meaningfully more expensive. Claude supports up to 200K tokens; the practical limit in any given application usually comes down to cost rather than the hard technical ceiling.

A well-documented failure mode worth knowing: the **“lost-in-the-middle” effect** — models pay measurably less attention to information buried in the middle of a long context than to information at the start or end, with accuracy drops of roughly 20-30% on mid-context retrieval. The practical strategy is not to stuff the context window just because you technically can — retrieve and include only what's actually relevant, which is the entire premise behind RAG (Q11).

8. What is an embedding and what are they used for?

An embedding is a dense, real-valued vector that represents text in a semantic space — texts with similar meaning map to nearby vectors (measured by cosine similarity — see Q9-Q10 for the normalization and mechanics), which is what powers semantic search, clustering, classification, and retrieval systems including RAG. Dimension matters less than you'd expect past roughly 768-1024 — quality gains beyond that are marginal for most retrieval tasks, though bigger, more expensive models (like OpenAI's text-embedding-3-large at 3072 dims) still edge out smaller ones on benchmarks like MTEB.

One operational gotcha: embedding drift — if you switch embedding models, old and new vectors are not compatible, and you must re-embed your entire corpus from scratch.

 **Your Implementation:** RAG/embed.py uses Bedrock Titan Text Embeddings v2, 1024-dim, normalize=True (unit-length vectors, ready for cosine similarity with no extra normalization step). It embeds only the **child** chunks produced by chunking.py — see Q41 for why parents and children are split.

9. What is cosine similarity?

Cosine similarity measures the angle between two vectors, not their magnitude, and is the fundamental operation that makes semantic search work. Given vectors **a** and **b** of dimension **d**:

$$\begin{aligned}\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) &= (\mathbf{a} \cdot \mathbf{b}) / (||\mathbf{a}|| \times ||\mathbf{b}||) \\ &= \sum(\mathbf{a}_i \times \mathbf{b}_i) / (\sqrt{\sum(\mathbf{a}_i^2)} \times \sqrt{\sum(\mathbf{b}_i^2)})\end{aligned}$$

The result is always between **-1** and **+1**: **+1** = vectors point in exactly the same direction (identical/highly similar meaning) - **0** = vectors are orthogonal (completely unrelated) - **-1** = vectors point in opposite directions (contradictory meaning)

Why it matters for retrieval: embeddings are typically normalized to unit length (see Q10 for the full normalization mechanics; RAG/embed.py does this with `normalize=True`), so for unit-length vectors the denominator becomes 1, and cosine similarity simplifies to a dot product $\mathbf{a} \cdot \mathbf{b}$. This matters because: (1) it's fast to compute at scale — you're just multiplying coordinates and summing, (2) it's scale-invariant — the angle between vectors is all that matters, not their magnitude, and (3) two documents with *similar meaning* will have vectors close together in the embedding space, even if they use completely different words (“car” vs. “automobile”). That semantic signal is exactly what dense retrieval (as opposed to BM25's exact keyword matching) provides.

Real example from your repo: a query like “how do I tag a git commit” and a document saying “use git tag to create a version marker” have no overlapping keywords — BM25 scores them poorly, possibly zero. But their embeddings, trained on millions of developer Q&A pairs, land close together in vector space, so cosine similarity retrieves them correctly. This is the fundamental mechanism enabling the paraphrase robustness measured in your retrieval eval (Q42).

 **Your Implementation:** RAG/faiss_search.py uses IndexFlatIP (Inner Product), which for unit-length vectors is mathematically identical to cosine similarity — it computes $\mathbf{a} \cdot \mathbf{b}$ for normalized vectors, which is the cosine similarity.

10. What are normalized embeddings?

Normalized embeddings are unit-length vectors — embeddings that have been divided by their L2 norm (magnitude) so they all have length 1.0. The mathematical transformation is:

$$\begin{aligned}\text{normalized_embedding} &= \text{embedding} / ||\text{embedding}|| \\ &= \text{embedding} / \sqrt{\sum(\mathbf{e}_i^2)}\end{aligned}$$

Why normalization matters:

1. **Simplifies cosine similarity to dot product:** For unit-length vectors, cosine similarity reduces to a simple dot product $a \cdot b$. The full formula's denominator ($\|a\| \times \|b\|$) becomes $1 \times 1 = 1$, so it drops out. This is why RAG/embed.py uses `normalize=True` — it precomputes this once during embedding, saving computation at search time.
2. **Enables efficient exact matching via inner product:** FAISS's `IndexFlatIP` computes dot products between vectors. For normalized vectors, this directly yields cosine similarity scores — no extra normalization step needed at query time. This is why your implementation uses `IndexFlatIP` rather than `IndexFlatL2` (Euclidean distance): the normalized vectors make inner product the right metric.
3. **Direction-only comparison:** Normalization ensures that only the *direction* of the vector matters for similarity, not its magnitude. Two semantically identical documents produce vectors pointing in the same direction regardless of “strength,” and normalization makes this property explicit in the math.
4. **Numerical stability:** Unit-length vectors have bounded magnitude, which helps prevent numerical overflow/underflow issues when doing large-scale vector operations.

Concrete example:

Raw embedding: `[0.5, 0.3, 0.2]` (magnitude = $\sqrt{0.25 + 0.09 + 0.04} \approx 0.62$)
Normalized: `[0.81, 0.49, 0.32]` (magnitude = 1.0)

When you compute `dot_product(normalized_a, normalized_b)`, the result is automatically between -1 and +1 — it *is* the cosine similarity.

Connection to cosine similarity (Q9): Cosine similarity *requires* normalized vectors to work efficiently in practice. Without normalization, you'd compute the full formula every time: $(a \cdot b) / (\|a\| \times \|b\|)$. Normalization pre-computes the division, storing vectors that are already unit-length, so similarity search only needs the dot product — no square roots, no divisions, just multiply-and-sum.

 **Your Implementation:** RAG/embed.py normalizes embeddings with the `normalize=True` flag in the Bedrock Titan v2 API call. Every embedding stored in `child_chunks_embeddings.jsonl` is already unit-length. When `faiss_search.py` loads them and builds `IndexFlatIP`, the inner-product-based search is mathematically identical to cosine similarity search — no extra normalization needed.

11. What is RAG (Retrieval-Augmented Generation)?

RAG combines a retriever — which searches a knowledge base — with an LLM generator: retrieved chunks are injected directly into the prompt, grounding the model's response in real data rather than relying purely on what it memorized during pretraining. Because the knowledge lives externally in a vector store rather than in the model's weights, no retraining is needed to keep the system current — you just update the index.

Query → Embed → Vector DB Search (top-k) → Prompt (context + query) → LLM Generate → Grounded Response

Core metrics: Recall@k (target >80%), Precision@k (>70%), MRR (>0.8) — see Q59 for how a real eval set measures these. RAG is the right tool for domain-specific or frequently-updated knowledge with citation needs; it's the wrong tool for common-sense reasoning or genuinely complex multi-hop questions, where agentic RAG (Q54) does better.

 **Your Implementation:** This repo's RAG/ folder is exactly this pipeline end to end — chunking.py → embed.py → faiss_search.py → hybrid_search.py → generate.py → api.py, measured at 90.9% hit rate / 0.909 MRR on an 11-question held-out eval set.

12. What is fine-tuning in LLMs?

Fine-tuning continues training a pretrained model on a smaller, domain-specific dataset to adapt its behavior. There are three main flavors: full fine-tuning (updates every weight, most expensive), PEFT methods like LoRA (Q81, updates a small added set of parameters), and instruction tuning (teaches the model to follow commands rather than just complete text). It's cheaper than pretraining from scratch, but it carries a real risk of catastrophic forgetting (Q84) — the model can lose prior general capabilities while adapting to the new domain.

Decision rule worth having ready: start with prompt engineering, since it's free and fast to iterate. Move to fine-tuning only after 2-3 prompt-engineering iterations plateau *and* you have 500+ high-quality examples. And don't expect fine-tuning to reliably teach new facts — that's a job for RAG (Q11); fine-tuning is much better suited to teaching style, format, and task-specific behavior.

13. What is the attention mechanism in a Transformer?

The Transformer architecture (high level): The Transformer is the neural network design that almost every modern LLM is built from. Its core trick is that it looks at *all* the words in your input at the same time and figures out which words should pay attention to which other words — all in one pass, in parallel. Before Transformers, models read text left-to-right, one word at a time, and the single “memory” for everything so far kept getting overwritten. The Transformer dumps the whole input on the table at once, lets every token look directly at every other token, and processes it all simultaneously. That’s why it’s fast and handles long-range connections well.

Attention is the “which words matter to me” step. Take the sentence *“The trophy didn’t fit in the suitcase because it was too big.”* To understand “it”, the model has to decide: does “it” mean the trophy or the suitcase? Attention lets the word “it” reach back, compare itself against every other word, and decide which one is most relevant. It then pulls in meaning from that word and mixes it into its own representation.

Mechanically it works like a soft search. Every token produces three learned projections:

- a **query** (Q): “here’s what I’m looking for”
- a **key** (K): “here’s what I’m about”
- a **value** (V): “here’s my content”

The query gets compared against all the keys, producing match scores. Those scores get turned into percentages using softmax (so they sum to 100%). Then the token receives a weighted blend of all the values, where the percentages are the weights. So “it” might pull 70% from trophy, 10% from suitcase, and the rest scattered elsewhere — never a hard yes/no, always a soft weighted mixture.

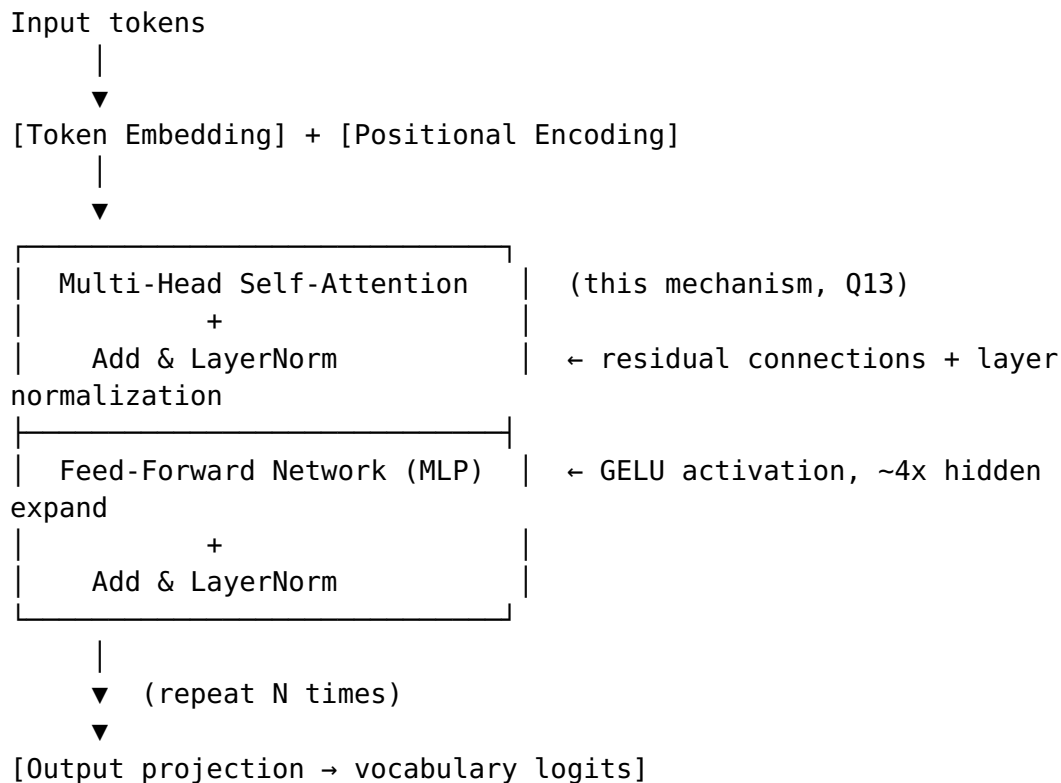
The formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Dividing by $\sqrt{d_k}$ keeps the dot products from exploding before softmax. This runs in $O(n^2)$ time — every token compares itself to every other token — which is why long context windows are expensive (Q7) and why optimizations like PagedAttention (Q102) exist.

Multi-head attention runs h parallel attention operations side by side, each with its own learned Q/K/V projections. They specialize: one head might track grammar, another pronouns and referents, another topical similarity. Their outputs concatenate, so each word ends up enriched from several angles at once.

The complete Transformer block, stacked N times to form the full model:



For a decoder model (GPT/Claude-style), the self-attention is causal/masked — each token can only attend to itself and previous tokens, never future ones (this is what makes autoregressive generation possible, Q16). For an encoder-decoder model (T5-style), the decoder adds a cross-attention sub-layer between self-attention and feed-forward, letting the decoder attend to the encoder’s output. This architecture scales remarkably well: more data plus more compute reliably yields better performance, which is the empirical basis for the scaling laws in Q18 and Q87. The $O(n^2)$ complexity of attention is why long context is expensive and why PagedAttention (Q102) exists.

14. What is greedy decoding vs beam search?

Greedy decoding picks the single highest-probability token at every step — it’s fast (one forward pass per token) but locally optimal, meaning it can miss a better overall sequence that would have required a lower-probability token somewhere along the way. **Beam search** instead keeps the top- B partial sequences (“beams”) at each step, expands each of them, and prunes back

down to the top B by cumulative score — this explores more of the possibility space and generally produces better quality, at the cost of B times more compute, and it's still not guaranteed to find the globally optimal sequence.

In practice, most production chat systems favor greedy decoding combined with sampling (Q15) rather than beam search — beam search tends to produce bland, repetitive text for open-ended generation even when it scores well on narrow automated metrics. Beam search remains genuinely useful for tasks with something closer to a single correct answer, like translation.

15. What is top-k and top-p (nucleus) sampling?

The setup first: the model never just “picks a word.” At every step it outputs a probability for *every single word in its vocabulary* — all ~100,000 of them. “Paris” might get 97%, “Lyon” 1%, and then there’s a long tail of 99,998 words holding tiny slivers of probability, including complete nonsense like “refrigerator.” If you sampled straight from that raw list, you’d occasionally roll one of those 0.0001% words and produce garbage. So before sampling, you cut the list down. Top-k and top-p are two different rules for where to make that cut.

Top-k keeps a fixed number of words. `top_k=40` means “throw away everything except the 40 most likely words, then sample from those.”

Top-p (nucleus) keeps a fixed amount of probability. `top_p=0.95` means “walk down the ranked list adding up probabilities until you reach 95%, then stop and throw away the rest.” The *number* of words kept changes at every step.

That difference is the whole point, and two examples make it obvious:

*“The capital of France is ____” — the model is very confident:

Word	Probability
Paris	97%
Lyon	1%
Nice	0.5%
...long tail	remainder

- **top_k=40** keeps Paris *plus 39 wrong cities* — a real ~3% chance of sampling one and announcing that the capital of France is Marseille.

- **top_p=0.95** notices Paris alone already covers 95%, keeps only Paris, and cannot get it wrong.

*“My favorite color is ____” — the model is genuinely uncertain, and it should be:

Word	Probability
blue	20%
red	15%
green	12%
purple	10%
black	8%

- **top_p=0.95** has to go 15-20 words deep to reach 95%, so it keeps a wide pool — exactly right, since you want variety here.

The takeaway: top_k uses the same size pool whether the model is certain or clueless. Top_p shrinks the pool when the model is confident and widens it when the model isn’t. That adaptivity is why top_p generally produces more coherent output and why it’s the more common default.

A common production pairing is top_k=40, top_p=0.95 — top_p does the real work, top_k acts as a hard ceiling safety net. One ordering detail that’s easy to get wrong in an implementation: temperature (Q3) is applied **first** — it reshapes the probabilities, flattening or sharpening them — **then** top_k/top_p cut the candidate list, **then** sampling happens from what survives. Filtering before applying temperature gives behavior nobody expects.

16. What is autoregressive generation?

Autoregressive generation produces tokens one at a time, each conditioned on every token generated before it:

$$P(x_1 \dots x_n) = P(x_1) \times P(x_2 | x_1) \times P(x_3 | x_1, x_2) \times \dots = \prod P(x_i | x_{<i})$$

This mirrors the model’s training objective directly — it was trained to predict the next token, so generation just repeats that prediction step sequentially. It’s only possible because decoder attention is *causal*, meaning it’s masked so token i can never see tokens that come after it (Q1b/Q175); this is the actual mechanical reason LLMs generate text the way they do, while an encoder model like BERT — which sees the whole input bidirectionally at once — cannot generate text this way at all.

Because each token depends on all previous ones, generation is inherently sequential and can't simply be parallelized across the output; speeding it up requires techniques like speculative decoding (Q106) rather than brute-force parallelism.

17. What is a system prompt?

A system prompt is a hidden instruction block, prepended before the user's messages, that sets the model's persona, constraints, tone, and safety boundaries for the whole conversation — it's how operators customize a general-purpose model like Claude for their specific product. A good system prompt typically establishes an identity ("You are a..."), a goal, explicit constraints, and the desired output format.

Important limitation: a system prompt is *not* a security boundary on its own. A sufficiently motivated user can often work around its constraints through prompt injection (Q24), so anything safety-critical needs an external verification layer that doesn't rely on the system prompt alone to prevent a bad outcome.

(Q18-20 are not present in the source document — it jumps from Q17 directly to Q21. Nothing has been invented to fill the gap.)

PROMPT ENGINEERING (Q21-40)

21. What is chain-of-thought (CoT) prompting?

Chain-of-thought prompting adds intermediate reasoning steps to a prompt — the classic trigger phrase is "let's think step by step" — which measurably improves accuracy on multi-step arithmetic, logic, and commonsense reasoning tasks by forcing the model to decompose the problem rather than jump straight to an answer. Zero-shot CoT relies on the phrase alone; few-shot CoT goes further and provides worked examples showing the reasoning steps explicitly.

Effectiveness varies sharply by task: roughly 40-50% improvement on math problems, 10-20% on general text understanding, and near-zero on tasks the model was already strong at (a ceiling effect). It costs roughly 2-3x more output tokens than a direct answer, so it's worth using when accuracy genuinely matters and wasteful for simple classification.

22. What is self-consistency prompting?

Self-consistency samples N diverse reasoning chains for the same question (typically at higher temperature) and takes a majority vote on the final answer, on the theory that errors in individual reasoning chains are largely uncorrelated, so voting cancels them out. It's effective but costs N times the inference — 5-10 samples is usually sufficient to see the benefit.

It only helps when the sampled reasoning paths are genuinely diverse; if temperature is too low, all N samples converge on the same (possibly wrong) path and voting adds cost without benefit. Reserve it for high-stakes tasks where the multiplied cost is justified.

23. What is ReAct (Reasoning + Acting) prompting?

ReAct interleaves reasoning traces with action calls — search, code execution, API calls — in a loop: the model thinks, acts, observes the result, and thinks again. This is the foundation of how modern AI agents interact with external tools (Q61), and it's what distinguishes ReAct from plain chain-of-thought: ReAct can access *current* information the model was never trained on, because it calls real tools mid-reasoning rather than reasoning purely from memorized knowledge.

 **Related to your implementation:** RAG/
query_transform.py's HyDE step (Q47) is a simpler, single-shot cousin of this idea — one LLM call to improve the search input, rather than a full multi-step ReAct loop.

24. What is prompt injection and how is it mitigated?

Prompt injection is an attack where malicious text — often embedded in retrieved data or user input — overrides the system's intended instructions. Mitigations should be layered, since no single defense is sufficient alone: input sanitization (weak on its own), privilege separation (clearly tagging user input separately from system instructions so the model learns to respect the boundary), fine-tuning the model to be robust to known injection patterns, and — most important for production — **external verification**: never let the model's own judgment be the sole gate on a consequential action like a money transfer or a data deletion; always add a code-level check that doesn't blindly trust the LLM's output.

25. What is few-shot learning vs in-context learning?

Few-shot learning places example input-output pairs directly in the context window to guide the model's behavior; in-context learning is the broader underlying phenomenon that makes this possible — the model adapts to a task from examples *without any gradient updates*, purely from what's in the prompt. In practice these terms are used almost interchangeably: few-shot prompting is the deliberate application of in-context learning, which is itself an emergent capability that shows up reliably only at sufficient model scale.

It helps most for structured outputs like classification and extraction, and helps least for creative generation or genuinely novel reasoning tasks that fall outside the model's pretraining distribution.

26. What is meta-prompting?

Meta-prompting uses an LLM to generate or refine prompts for another LLM task, automating part of the prompt engineering process itself and reducing manual iteration — models can, in effect, self-improve their own prompts. It's genuinely useful for R&D and exploration, surfacing non-obvious prompt phrasings faster than manual iteration would. For production use, though, hand-crafted and tested prompts are usually still preferred, since LLM-generated prompts can be needlessly verbose or brittle in ways that only become obvious when they fail on an edge case.

27. What is role prompting?

Role prompting assigns a persona to the model — “You are a senior software engineer” — which noticeably shapes tone, apparent knowledge depth, and problem-solving style. It's especially effective for domain-specific tasks like legal or medical Q&A, where the right register and vocabulary matter. It's worth being precise about what it does and doesn't do: it has a real effect on tone and depth, but a much more limited effect on actual reasoning capability — role prompting doesn't make the model smarter, it shifts the register it draws from. Don't rely on it alone for accuracy gains on hard reasoning tasks; combine it with chain-of-thought (Q21) for that.

28-30. (not present in the source document — see note at top)

31. What is structured output prompting?

Structured output prompting instructs the model to respond in a strict format — JSON, XML, or similar — usually combined with function calling, so that downstream code can parse the output reliably. This is essential for any production pipeline where an LLM’s output feeds directly into other code, rather than being read by a human. Modern models increasingly support constrained generation (grammar-based decoding) that *guarantees* valid JSON structurally rather than just hoping the model follows format instructions — worth reaching for when malformed output would crash your pipeline.

32. What is a prompt template?

A prompt template is a reusable prompt structure with variables filled in at runtime, standardizing LLM inputs across a pipeline and improving consistency between requests. Frameworks like LangChain and LlamaIndex provide template management out of the box. A practical point worth remembering: version your templates like code — a silent, undocumented prompt template change is one of the most common causes of unexplained quality regressions in production, so track template versions alongside your eval scores to be able to bisect when quality drops.

33. What is automatic prompt optimization (APO)?

APO uses an optimizer — gradient-based or LLM-based — to improve prompts automatically rather than through manual iteration, which becomes critical for complex multi-step tasks where hand-tuning every prompt doesn’t scale. DSPy is the leading framework here: it compiles a high-level task description into an optimized prompt program, and its “teleprompter” component automatically searches for the best few-shot examples and instructions for a given metric — conceptually similar to hyperparameter search, but applied to prompts instead of model parameters.

34. What is context window management in production?

This is the practice of selecting and prioritizing what actually fits within a request’s token limits — via summarization, sliding windows, retrieval, or priority scoring — since poor context management is a top cause of production LLM failures in real systems. It’s worth recognizing that this is precisely the problem

RAG’s retrieval step solves systematically: rather than manually managing a sliding window over raw conversation history, you retrieve only the top-k relevant pieces for each request.

 **Related to your implementation:** your RAG/hybrid_search.py score threshold caps context to only genuinely relevant chunks instead of padding the window to fill it — a concrete instance of this principle (see Q45).

35. What is prompt chaining?

Prompt chaining breaks a complex task into a sequence of simpler prompts, where each step’s output feeds the next — each individual step is easier to get right and more reliable than one giant “mega-prompt” trying to do everything at once, and it enables auditing, debugging, and human review at each point in the chain. The trade-off worth stating in an interview: chaining trades latency (multiple sequential LLM calls) for reliability and debuggability.

 **Related to your implementation:** RAG/generate.py is technically a two-step chain already — retrieve, then generate — with an explicit refusal gate between the steps (Q56) rather than one monolithic prompt trying to do retrieval and generation at once.

36. What is step-back prompting?

Step-back prompting asks the model a higher-level, more abstract question before tackling the specific one, retrieving relevant background knowledge that then informs reasoning on the actual task. This measurably improves accuracy on physics, history, and other complex reasoning benchmarks that benefit from establishing general principles first. It mirrors how a human expert typically approaches an unfamiliar problem — establish the governing principle, then apply it to the specifics — at the cost of one extra LLM call.

37. What is Tree-of-Thought (ToT) prompting?

Tree-of-thought prompting explores multiple reasoning paths as a branching tree, evaluating and pruning weaker branches as it goes — more powerful than linear chain-of-thought for tasks that require search or planning, since it can abandon a bad path mid-reasoning rather than only discovering it was wrong at the very end. It requires meaningfully more inference calls than CoT, but solves harder planning/search problems more reliably as a result.

Think of the progression as: CoT is a single reasoning line; self-consistency (Q22) is N independent parallel lines voted at the end; ToT is a branching tree that can prune bad paths *during* reasoning, not just after. Reserve it for genuinely hard planning problems, not routine tasks where the extra cost isn't justified.

38-40. (not present in the source document — see note at top)

RAG & RETRIEVAL (Q41-60) — Your Core Learning Area

41. How does RAG work end-to-end?

RAG runs in two phases. **Offline (indexing)**: documents are chunked, each chunk is embedded into a vector, and the vectors are stored in a vector database along with metadata. **Online (query time)**: the user's query is embedded with the same model, an approximate nearest-neighbor (ANN) search finds the top-k most similar chunks, those chunks are injected into the LLM's prompt, and the LLM generates a grounded answer that cites the retrieved sources.

📌 **Your Implementation — the real architecture in this repo, and a genuinely more sophisticated variant of the pipeline above:**

```
Markdown notes (data/*.md)
  ▼
chunking.py → PARENT chunks (chunks.jsonl, ≤700 tok,
header-breadcrumbed,
  |               never split mid-code-fence)
  |               + CHILD chunks (child_chunks.jsonl, ≤150
tok, parent_id
  |               linked, 15% overlap)
  ▼
embed.py → embeds CHILDREN only (Bedrock Titan v2, 1024-
dim, normalized)
  ▼
faiss_search.py --build → IndexFlatIP (exact cosine
search, brute-force
  |                               by design at this scale)
  ▼
query_transform.py → HyDE: rewrite query as a hypothetical
answer (Q47)
  ▼
hybrid_search.py → BM25 + FAISS → RRF fusion → rerank
```

→ promote the
|
PARENT chunk
▼
generate.py → grounded answer, numbered citations, refuses
if ungrounded
▼
api.py → FastAPI: POST /ask, GET /health

This is the **parent-child (“small-to-big”) pattern**: small chunks (150 tokens) are what get matched — precise, not diluted by unrelated content in a larger chunk — but the parent’s full 700-token section is what’s actually shown to the LLM, so it still has enough surrounding context to answer well. This is the honest answer to the classic follow-up “what chunk size do you use?” — two sizes, for two different jobs, not one number. Full design rationale: RAG/docs/parent-child-retrieval.md.

42. What is the difference between dense and sparse retrieval?

Sparse retrieval (BM25) does exact lexical/term matching via TF-IDF-style scoring — fast, interpretable, and great for specific terms or IDs, but it misses paraphrases and synonyms entirely (“car” and “automobile” don’t match at all). **Dense retrieval** (DPR, E5, and similar neural embedding models) compares learned vector representations via cosine similarity (Q9), so it handles paraphrase and synonymy naturally, at the cost of being slower and harder to interpret (“why did this match?” is much less obvious than with BM25).

In most published benchmarks, hybrid retrieval — combining both signals via **Reciprocal Rank Fusion (RRF)** — beats either alone. RRF merges by *rank*, not raw score, because BM25 scores and cosine similarities live on entirely incomparable scales; naively averaging the raw numbers produces meaningless results, a common interview trip-up.

📌 **Your Implementation:** RAG/hybrid_search.py runs both BM25 and dense retrieval and fuses with RRF — you don’t pick one. Concrete proof this matters, from your own measured data (docs/reranking-strategies.md): the same correct chunk scored **+4.90** for a near-exact-keyword query (“git tag”) but only **+0.53** for a paraphrased query asking the same thing — real, measured evidence of the dense-retrieval paraphrase gap this question is testing.

43. What is a vector database?

A vector database stores high-dimensional embeddings and supports approximate nearest-neighbor (ANN) search over them — it's the core infrastructure component behind any RAG or semantic search system. Common choices include Pinecone (managed, easy to start), Weaviate (full-featured, self-hostable), Qdrant (fast, Rust-based), Chroma (lightweight, developer-friendly), and pgvector (a Postgres extension, useful if you already run Postgres). "Approximate" is the operative word for most of these at meaningful scale: algorithms like HNSW or IVF trade a small amount of recall for large speed gains once you're past tens of thousands of vectors.

📌 **Your Implementation:** You use **local FAISS** (`faiss_search.py`, `IndexFlatIP`) — exact, brute-force search, not approximate. That's a deliberate choice at your current scale, since approximate indexes only pay off once brute-force search is actually measurably slow — see `docs/vector-store-strategies.md`. It also means no managed vector DB cost.

44. What is chunking strategy in RAG and why does it matter?

Documents can be split by fixed size, by sentence boundaries, or semantically (by topic shift); poor chunking breaks context across chunk boundaries and measurably degrades retrieval quality, which is why overlap between adjacent chunks is standard practice — it prevents information from being lost right at a split point.

📌 **Your Implementation:** `RAG/chunking.py` is structure-aware, not naive fixed-size — it parses markdown headers into a breadcrumb path (e.g. `["System Design Notes", "Load Balancing", "Layer 4 vs Layer 7"]`), merges small sibling sections together, and splits oversized sections at paragraph boundaries with an explicit `FENCE_RE` guard so it **never splits inside a fenced code block**. That guard is the practical difference between "I've read about chunking" and "I've hit the bug where chunking breaks a code block in half and ruins retrieval."

45. What is reranking in RAG pipelines?

A cross-encoder model reorders the retrieved chunks by relevance to the query after the initial fast retrieval pass — first-stage retrieval (ANN search) is fast but comparatively crude, while the reranker is slower but significantly more accurate, since it looks

at the query and each candidate chunk *together* rather than comparing precomputed independent vectors. This meaningfully reduces hallucination by ensuring the chunks actually shown to the LLM are truly relevant, not just approximately similar.

📌 **Your Implementation:** `hybrid_search.py` implements **two swappable reranker backends**: `local` (a sentence-transformers CrossEncoder, the default — free and runs on-device) and `bedrock` (Amazon Bedrock Rerank — implemented but currently blocked by an AWS Organizations SCP on this account, wired up behind the `TOOL_RAG_RERANK_BACKEND` env var so switching back is a one-line change once access is granted). It also enforces a **score threshold**, so a query with nothing genuinely relevant returns *fewer* results instead of being padded with confidently-wrong ones.

46. What are the trade-offs between fine-tuning and RAG?

Fine-tuning bakes knowledge directly into the model's weights — fast inference, no retrieval lag, but costly to update since any new information requires retraining. RAG keeps knowledge external and updatable — no costly retraining when facts change, at the cost of retrieval latency on every request. In practice, hybrid architectures (fine-tune *and* RAG together) are the dominant production pattern: fine-tune for style and output format, RAG for facts. A common and costly mistake is trying to fine-tune *new facts* into a model — it doesn't work reliably and risks catastrophic forgetting (Q84) for a knowledge-update problem RAG solves better and more cheaply.

47. What is HyDE (Hypothetical Document Embeddings)?

HyDE generates a hypothetical answer to the query first, then embeds *that* hypothetical answer for retrieval instead of the raw query — the reasoning being that a generated answer is phrased more like the actual documents in the corpus than a terse question is, so its embedding lands closer to the real matching chunk. This meaningfully improves recall especially for abstract or keyword-poor queries where the wording gap between question and answer is largest.

📌 **Your Implementation:** `RAG/query_transform.py::generate_hyde_document()` uses Claude Haiku 4.5 via Bedrock (the eu.-routed cross-region inference profile). A subtlety from your own code comments that most textbook answers miss: **BM25 still searches on the raw**

query — only the vector-search leg uses the HyDE rewrite, because HyDE fixes an embedding-similarity problem specifically, not search in general.

51. What is multi-vector retrieval?

Multi-vector retrieval stores multiple embeddings per document — for instance a summary embedding plus a full-document embedding plus proposition-level embeddings — rather than a single vector per chunk. ColBERT takes this further with token-level embeddings for “late interaction” retrieval, comparing individual token vectors rather than one pooled document vector. This improves recall for complex queries that a standard dense bi-encoder misses, at the cost of significantly more storage and search complexity.

Your repo’s parent-child chunking (Q44) is a lighter-weight relative of this same idea — two representations per document (a small precise child for matching, a large parent for context) rather than ColBERT’s much finer token-level granularity.

52. What is query transformation in RAG?

Query transformation rewrites, expands, or decomposes the user’s query before retrieval to reduce the vocabulary mismatch between how users ask questions and how the indexed documents are actually phrased. Multi-query is one common approach — generate N variant phrasings of the query, retrieve for each, and merge the results. HyDE (Q47) is another specific technique in this family. Your repo’s docs/query-transformation-strategies.md documents why HyDE specifically was chosen over alternatives like multi-query fan-out and query decomposition for this project.

53. What is metadata filtering in RAG?

Metadata filtering restricts retrieved documents by structured fields — date, author, category — before ranking, combining ANN vector search with SQL-like constraints for more precise retrieval. This is essential for enterprise RAG where per-document access control is required. Worth noting beyond the original point: in multi-tenant systems, metadata filtering is also a *security* mechanism, not just a relevance improvement — filtering by tenant/user ID before ranking is what prevents one customer’s query from ever surfacing another customer’s documents, something similarity search alone cannot guarantee.

54. What is iterative / agentic RAG?

In agentic RAG, an agent retrieves, reads the results, decides whether more retrieval is needed, and loops — handling multi-hop questions that require reasoning across multiple documents in a way single-shot RAG can't. Frameworks like LangGraph and LlamaIndex Agents orchestrate these retrieval loops. This repo's current pipeline is single-shot retrieve-then-generate (Q41), not agentic — a natural next evolution once you want to handle multi-hop questions your current eval set doesn't test for. The trade-off worth stating: agentic RAG handles harder questions but costs multiple LLM calls per query and carries a real risk of infinite loops without a hard iteration cap.

55. What is embedding model selection for RAG?

Model selection should be driven by MTEB benchmark scores for your specific task and language, with top models including OpenAI's text-embedding-3-large, E5-Mistral, and BGE-M3; domain-specific fine-tuning of the embedding model can further boost retrieval recall for specialized corpora. In practice, dimension count (cost/storage), latency, and licensing (open vs. API-gated) matter alongside the raw MTEB score. Your repo uses Bedrock Titan v2 at 1024 dimensions — a reasonable middle ground rather than the top MTEB scorer, chosen to avoid adding a second vendor dependency alongside Bedrock's generation calls.

56. What is faithfulness vs. relevance in RAG evaluation?

Faithfulness asks whether the generated answer is actually grounded in the retrieved context — no hallucination beyond what the sources support. **Relevance** asks whether the retrieved chunks themselves are actually on-topic for the query. The RAGAS framework is the standard tool for measuring both automatically as part of RAG pipeline evaluation.

📌 **Your Implementation — this repo does something stronger than “measure it with RAGAS”:** `RAG/generate.py` **enforces** both structurally rather than only measuring them after the fact. A **relevance gate**: if `search()` returns `[]` (nothing passed the rerank threshold), it refuses *before* calling the LLM at all — zero wasted generation calls on ungrounded questions. And **faithfulness enforcement**: the prompt requires numbered citations ([1], [2]) for every claim, citations are validated post-hoc against the real source count (out-of-range numbers are flagged, never

silently hidden), and a fixed refusal phrase is detected by substring match — so refusal is checkable in code, not just visible to a human reading the output.

57. What is the document ingestion pipeline for RAG?

The standard pipeline is: load → parse (PDF, HTML, DOCX) → clean → chunk → embed → index → store, handling tables, images, and code blocks separately for better extraction quality. Unstructured.io, LlamaParse, and Docling are popular parsing libraries for this.

 **Your Implementation:** RAG/chunking.py (see Q41/Q44 for the full mechanism) is specifically header-aware for markdown, rather than a general PDF/HTML/DOCX pipeline — the right scope for your current source data, though it would need extending (with something like Unstructured.io or Docling) if you start ingesting PDFs or Word documents later.

58-60. *(not present in the source document — see note at top)*

AI AGENTS (Q61-80)

61. What is an AI agent?

An AI agent uses an LLM as a reasoning engine inside a loop: perceive → think → act → observe. Unlike a single LLM call, an agent handles multi-step tasks and uses tools along the way, with frameworks like LangGraph, AutoGen, CrewAI, and the OpenAI Assistants API providing the orchestration layer. The clean distinguishing test versus a workflow (Q74): a workflow runs the same fixed steps every time, while an agent's LLM decides the next step dynamically based on what it's observed so far. By this definition, this repo's RAG pipeline (Q41) is currently a workflow, not an agent.

62. What is the Model Context Protocol (MCP)?

MCP is an open standard, created by Anthropic, for connecting AI systems to external tools and data sources — it defines a standard interface so any MCP-compatible tool plugs into any MCP-

compatible host, the way USB replaced bespoke device-specific connectors with one universal standard. Before MCP, connecting an LLM application to N different tools meant writing N different bespoke integrations; MCP standardizes the interface so a tool built once can be plugged into any compatible host.

63. What is tool use / function calling in agents?

An agent calls external tools — web search, code execution, database queries, APIs — by having the LLM output a structured tool call, which the host application then executes and returns the result of. This enables agents to act in the real world beyond pure text generation. A critical detail: the LLM never executes anything itself; it only *decides* which tool to call with what arguments, and this separation is an important safety boundary, since the host can validate, sandbox, or reject a tool call before it actually runs.

64. What are multi-agent systems?

Specialized agents — planner, retriever, coder, critic, and so on — collaborate by passing messages, with an orchestrator delegating subtasks and agents returning results back to the pipeline. Frameworks like AutoGen, CrewAI, and MetaGPT support building these multi-agent workflows. It's worth being able to justify *why* multiple specialized agents beat a single well-prompted agent for a given task, rather than reaching for multi-agent architecture by default — it adds real overhead in extra LLM calls, more failure surface, and harder debugging.

65. What are the main failure modes of AI agents?

Three worth knowing: **error propagation** (one agent's mistake cascades through the rest of the pipeline), **infinite loops** (agents ping-pong without making progress, requiring explicit timeout logic to prevent), and **prompt injection via the environment** (malicious content encountered during a task hijacks the agent's subsequent actions). Of these, always designing agent loops with a hard iteration cap and a cost/token budget ceiling is the single most important production safeguard — an ungoverned agent loop is a real, not theoretical, cost and safety risk.

66. What is the difference between reactive and planning agents?

Reactive agents respond to immediate observations without any long-term planning. **Planning agents** (also called Plan-and-Execute) generate a full plan upfront, then execute it step by step, which generally handles complex multi-step goals more reliably than a purely reactive approach. Reactive agents are cheaper and faster for simple tasks; planning agents cost an extra upfront LLM call to produce the plan but avoid the wasted work that can come from short-sighted reactive decisions on genuinely multi-step goals.

67. What is memory in AI agents?

Three layers: **short-term** memory is simply the conversation history held in the context window for the current session; **long-term** memory is a persistent store (a vector DB or regular database) recalled across sessions; **procedural** memory captures learned behaviors or skills stored for reuse across different tasks. Long-term memory, mechanically, is essentially RAG (Q11) applied to the agent's own history — the same embed-store-retrieve mechanics, just with past interactions as the corpus instead of documents.

68-70. *(not present in the source document — see note at top)*

71. What is an agent scratchpad?

The scratchpad is intermediate working memory where an agent records its reasoning steps as it works — ReAct agents (Q23) specifically store thought → action → observation triples there as they loop. This provides transparency and, critically, enables debugging of the agent's decision chain: without a scratchpad, you only see the final output and have no way to diagnose *why* the agent made a wrong decision partway through a multi-step task.

72. What is a tool manifest / tool schema?

A tool schema is JSON that describes an available tool's name, description, parameters, and types, which the LLM uses to decide when and how to call each tool correctly. The line worth remembering here is that good tool descriptions are as important as the tools themselves — a well-implemented tool with a vague or

ambiguous description gets called incorrectly, or not at all, since the natural-language description is effectively documentation the model reads to make its decision.

73. What is code execution as a tool?

Here, the agent writes code, executes it in a sandbox, reads the output, and iterates — enabling precise computation, data analysis, and file manipulation that a language model’s own “mental math” can’t reliably deliver. Both OpenAI’s Code Interpreter and Claude Code implement this pattern. This is the standard fix for LLMs being unreliable at exact arithmetic (Q19): rather than trusting the model’s internal math, have it write and run code to compute the exact answer. Sandbox isolation is non-negotiable — never execute agent-generated code with unrestricted system access.

74. What is the difference between agents and workflows?

Workflows are a fixed sequence of LLM and tool calls — deterministic pipelines where the steps are known in advance. Agents are dynamic — the LLM decides the next step based on current context, adapting as it goes. Use workflows when the process is genuinely known and use agents when it needs to adapt; workflows are cheaper, more predictable, and easier to test, so default to one whenever the process actually is fixed. This repo’s RAG pipeline (Q41) is a workflow by exactly this test.

75. What is LangGraph and why is it popular for agents?

LangGraph is a graph-based framework for building stateful, multi-actor agentic applications, where nodes represent processing steps and edges represent transitions between them — giving explicit control flow rather than implicit logic buried in a large prompt. It natively supports cycles, branching, and human-in-the-loop checkpoints. The explicit-control-flow property is the real selling point over a plain agent loop: a graph makes the possible paths through your agent visible and testable, which matters a great deal once an agent has more than 2-3 possible next steps.

76. What is an orchestrator vs sub-agent?

An orchestrator is a high-level planner that delegates tasks to specialized sub-agents, each focused on one capability domain — this separation of concerns makes complex agentic systems more maintainable, in the same way splitting a monolith into microservices does, at the cost of added coordination and inter-agent communication overhead.

77. What is agent evaluation and how is it done?

Agent evaluation should measure task completion rate, tool call accuracy, and trajectory efficiency — not just whether the final answer was correct. LLM-as-judge is commonly used to score intermediate steps, not only the final output, and eval datasets should be built from real user sessions so you can test for regressions on every release. The “not just final output” point is the crux: an agent can reach the correct final answer via an unreliable or expensive path (wrong tool calls, wasted loops), which a final-output-only eval would never catch.

 **Related to your implementation:** RAG/eval/ does the equivalent for retrieval specifically — hit rate and MRR *per question* (see eval/results/*.json), not just an aggregate pass/fail — the same “look past the final number” principle, applied to a ranking problem rather than a full agent trajectory.

78-80. (not present in the source document — see note at top)

TRAINING & FINE-TUNING (Q81-100) — Your Core Learning Area

81. What is LoRA (Low-Rank Adaptation)?

LoRA freezes the original weight matrix W and adds small trainable low-rank matrices B and A alongside it: $y = Wx + BAx$, where B is $(d_{\text{out}} \times r)$ and A is $(r \times d_{\text{in}})$, with rank r typically 8-64. Only B and A are trained — this reduces trainable parameters from millions or billions down to thousands, while $r=8-16$ typically captures enough task-relevant variation for most fine-tuning needs.

The memory impact is the headline number: full fine-tuning a 7B model needs roughly 56GB of GPU memory (model weights + gradients + optimizer states combined); LoRA at $r=8$ needs roughly 15GB — a 3.7x reduction — for 95-98% of full fine-tuning's quality. QLoRA (Q85) pushes this further by quantizing the frozen base model to 4-bit.

A natural next project for this repo: LoRA fine-tune a small model on a narrow task using your existing markdown notes as training signal — a direct continuation of the RAG work already built here.

82. What is PEFT (Parameter-Efficient Fine-Tuning)?

PEFT is the umbrella term for methods that update only a small subset of a model's parameters while freezing the rest — variants include LoRA, prefix tuning, prompt tuning, IA³, and adapter layers, and the approach enables multi-task serving via swapping different adapters on top of a single frozen base model. LoRA is the industry sweet spot among these: roughly 0.05% of parameters trained for 95-98% of full fine-tuning quality. Prompt tuning trains even fewer parameters ($\sim 0.01\%$) but quality drops more noticeably (85-90%), useful mainly when GPU memory is extremely constrained.

83. What is DPO (Direct Preference Optimization)?

DPO re-frames RLHF alignment as a single supervised training objective, removing the need for a separate reward model entirely: given preference pairs of (chosen, rejected) responses, it trains directly with a binary cross-entropy-style loss that maximizes the likelihood ratio of the chosen response relative to a frozen reference model, while minimizing it for the rejected response. This makes it simpler than PPO-based RLHF (Q92), more stable to train, and competitive in quality — which is exactly why DPO gained adoption over the more complex PPO pipeline.

84. What is catastrophic forgetting in LLM fine-tuning?

Fine-tuning can overwrite pretrained knowledge — the model “forgets” prior general abilities while adapting to new, narrower data. Mitigations, ranked by real-world effectiveness: LoRA/PEFT (freezing the base weights entirely is the strongest defense, since general knowledge literally can't be overwritten if it's frozen), mixing a portion of the original pretraining-style data back into

the fine-tuning set (~80/20 new/old), early stopping, using a smaller learning rate, and EWC regularization (theoretically sound — it explicitly penalizes changes to weights important for previous tasks — but rarely used in practice due to its cost). Full fine-tuning can lose 20-50% of general knowledge in severe cases; LoRA is far more resistant precisely because the base weights never change.

85. What is QLoRA?

QLoRA quantizes the base model down to 4-bit precision and applies LoRA adapters on top in higher precision (typically fp16), enabling fine-tuning of 65B+ parameter models on a single 48GB GPU — a massive cost saving over full-precision fine-tuning, with near-full fine-tuning quality (roughly 93-96%, slightly below plain LoRA's 95-98%, since the base weights are now quantized too). Concretely, a 7B model shrinks from ~14GB (fp16) to ~3.7GB total. One real limitation: you can't cleanly merge QLoRA adapters back into the base model afterward, since quantization is irreversible — you deploy it as a separate adapter rather than a single merged model.

86. What is the pretraining pipeline of an LLM?

The pipeline runs: data collection → cleaning/deduplication → tokenizer training → distributed training → evaluation, typically over trillions of tokens across weeks on thousands of GPUs, with the Chinchilla scaling law (Q87) giving the rule of thumb that optimal model size scales with roughly training-tokens/20. This is a stage almost no individual engineer ever does from scratch in practice — it's the domain of foundation model labs (Anthropic, OpenAI, Meta). What an AI engineer will actually do day-to-day is fine-tuning (Q12/Q81) on top of an already-pretrained model, a distinction worth being explicit about rather than conflating the two in an interview.

87. What are Chinchilla scaling laws?

Given a fixed compute budget C , the compute-optimal model size scales as $N^* \propto \sqrt{C}$ and the optimal number of training tokens scales as $D^* \propto \sqrt{C}$ as well — in practical terms, this works out to roughly **20 training tokens per parameter**. Chinchilla-optimal training for a 70B-parameter model is therefore about 1.4T tokens, not simply “more parameters” for a fixed budget.

The insight worth being able to explain clearly: most pre-2022 models (GPT-3, OPT) were significantly *undertrained* relative to their parameter count. Chinchilla — and later LLaMA, which applied the same philosophy — showed that a smaller model trained on proportionally more tokens can beat a larger, undertrained model at the *same* total compute budget, which also delivers better inference efficiency (a smaller model is cheaper to run) as a bonus. That’s the actual reasoning behind “smaller model, more tokens” as a design philosophy.

88-90. (not present in the source document — see note at top)

91. What is SFT (Supervised Fine-Tuning)?

SFT is the first phase of alignment: fine-tuning a base LLM on curated instruction-demonstration pairs, teaching it output format, dialogue structure, and basic instruction-following before any preference optimization happens. It’s the foundation the rest of the alignment pipeline depends on — without good SFT, downstream reward model or DPO (Q83) training tends to fail, since the model hasn’t yet learned basic instruction-following well enough to give preference optimization something stable to refine.

92. What is the PPO algorithm in RLHF?

Proximal Policy Optimization trains the policy (the LLM) by optimizing a clipped probability ratio against a reward signal, with a KL-divergence penalty $\lambda \cdot \text{KL}(\pi_\theta || \pi_{\text{SFT}})$ that keeps the model from drifting too far from its SFT starting point during optimization. Real challenges in practice include reward hacking (the model finds ways to maximize reward that don’t actually reflect genuine quality), mode collapse, and difficulty calibrating the KL penalty correctly. These recurring pain points are exactly why DPO (Q83) gained adoption — it delivers competitive results with a much simpler, more stable supervised training objective.

93. What is adapter merging and Task Arithmetic?

Define a task vector as $\tau = \theta_{\text{fine_tuned}} - \theta_{\text{base}}$ — the direction fine-tuning moved the weights for a specific task. Task Arithmetic merges multiple such vectors by simple addition ($\theta_{\text{base}} + \tau_A + \tau_B + \dots$) to get a model competent at multiple tasks at once, without ever jointly training on all of them — enabling essentially zero-cost multi-task deployment. It doesn’t always work cleanly

when task vectors conflict on a given weight's direction; TIES merging specifically resolves these sign conflicts among task vectors for better combined quality.

94. What is continual learning in LLMs?

Continual learning trains on a sequence of tasks or domains over time without forgetting knowledge from earlier ones — the central obstacle is catastrophic forgetting (Q84) under standard gradient descent. Approaches include replay buffers (mixing old task data back in), EWC regularization, and modular adapter systems. In practice, modular adapter systems — one LoRA adapter per task or domain, swapped in at inference time — are by far the most common production approach, since they sidestep catastrophic forgetting entirely by never actually merging the weights, at the cost of needing to know which adapter to load for a given request.

95. What is data curation for LLM fine-tuning?

Quality beats quantity decisively: 1,000 high-quality examples often outperforms 100,000 noisy ones. Curation means deduplicating, filtering, and balancing the dataset — removing redundant, toxic, or low-signal data. Common open instruction-tuning datasets include Alpaca, ShareGPT, and UltraChat.

A concrete path for this repo: your markdown notes corpus (RAG/data/*.md) is a natural seed for a future instruction dataset, but raw notes aren't instruction pairs on their own — you'd need to generate (question, grounded-answer) pairs from them first, similar to how RAG/eval/retrieval_eval_set.json was already built for evaluating retrieval. That's the realistic first step toward an actual fine-tuning pipeline here.

96. What is the role of learning rate in fine-tuning?

Learning rate has to be small enough to preserve pretrained knowledge while still adapting — too high causes catastrophic forgetting (Q84), too low and there's effectively no adaptation from the fine-tuning at all. A warmup-then-cosine-decay schedule is standard for transformer fine-tuning. The right range genuinely depends on method, which is worth being precise about: **full fine-tuning** typically uses $1e-5$ to $1e-4$ (conservative, since every weight is being touched); **LoRA** can go noticeably higher, $1e-4$ to $5e-4$, since far fewer parameters are being updated and there's less risk of disrupting the frozen base; **QLoRA** sits in a similar

range to LoRA. A useful diagnostic regardless of method: if training loss *increases* after the first epoch, the learning rate is too high — that's your practical signal to back off.

97. What is multi-task fine-tuning?

Multi-task fine-tuning trains simultaneously on multiple task datasets, which improves generalization and prevents overfitting to any single task, broadening instruction-following ability. T5 and Flan-T5 pioneered this approach, with the Flan collection spanning 1,800+ distinct fine-tuning tasks. The trade-off against Task Arithmetic (Q93): multi-task fine-tuning trains everything jointly upfront, which is more expensive but can find genuine cross-task synergies that separate training followed by merging cannot discover.

98-100. (not present in the source document — see note at top)

INFERENCE & SERVING (Q101-120)

101. What is KV caching in LLM inference?

KV caching stores the key and value tensors computed for previous tokens so they don't need to be recomputed on every subsequent generation step — this reduces per-step compute from $O(n)$ down to $O(1)$ for cached prefix positions, though the cache's memory footprint grows linearly with both sequence length and batch size.

 **Your Implementation — a real, documented gap:** RAG/api.py wraps generate.py in FastAPI (POST /ask, GET /health), but per docs/api-strategies.md, the FAISS index and parent chunks are **not yet cached across requests** — every call re-loads them from disk. This is a live, concrete example for this exact question: an actual measured performance gap you can point to and fix, not a hypothetical “what would you optimize.”

102. What is PagedAttention?

PagedAttention manages the KV cache the way an operating system manages virtual memory — using fixed-size pages or blocks rather than one large contiguous allocation. This gives

near-zero memory fragmentation and enables memory sharing across parallel sampling or beam search, and it's the core innovation behind vLLM (Q113), delivering a 2-4x throughput improvement over naive serving approaches. The OS-virtual-memory analogy is worth leading with in an interview: just as an OS pages memory to avoid fragmentation from variable-size allocations, PagedAttention pages the KV cache to avoid fragmentation from variable-length sequences sharing GPU memory.

103. What is continuous batching in LLM serving?

New requests join the batch mid-flight as soon as other sequences in the batch finish, rather than waiting for the entire static batch to complete together — this eliminates idle GPU time and, combined with PagedAttention (Q102), dramatically increases GPU utilization. Without it, naive static batching means a batch of 8 requests all wait for the *slowest* request to finish before any new request can join, which continuous batching directly fixes.

104. What is model quantization?

Quantization reduces weight precision — typically FP32 down to INT8 or INT4 — for a smaller memory footprint and faster inference. GPTQ is a one-shot post-training quantization method using Hessian-based error minimization to achieve near-FP16 quality; AWQ preserves the most salient weights specifically, and enables running 70B-parameter models on a single 48GB GPU. Worth not conflating with QLoRA (Q85): quantization here is a post-training technique for deploying an already-trained model cheaply, whereas QLoRA quantizes specifically to make *training* cheaper on a quantized base.

105. What are TTFT and TPS metrics in LLM serving?

TTFT (time-to-first-token) is dominated by the prefill phase and is the key latency metric users actually feel. **TPS** (tokens-per-second) is dominated by the decode phase and is the throughput metric that matters for overall generation speed. Disaggregating prefill and decode across separate GPU pools lets you optimize each independently, since they have fundamentally different hardware bottlenecks (Q116) — prefill is compute-bound, decode is memory-bandwidth-bound — and optimizing for one on shared hardware often hurts the other.

106. What is speculative decoding?

A small, fast “draft” model proposes k candidate tokens ahead of time; the large target model then verifies all of them in a single forward pass, keeping accepted tokens and correcting the first rejected one. This delivers a 2-4x speedup, ideal for long outputs, with **zero quality degradation** — a detail worth stating precisely, since the target model’s output distribution is mathematically unchanged: it *verifies* rather than trusts the draft model’s proposals, and any rejected token is corrected by the target model itself. This contrasts with quantization (Q104), which does trade some quality for speed.

107. What is tensor parallelism vs pipeline parallelism?

Tensor parallelism splits individual weight matrices across multiple GPUs, requiring an all-reduce communication step at every layer — this needs fast GPU-to-GPU interconnect (like NVLink) and is typically used within a single node. **Pipeline parallelism** instead partitions entire layers across different devices, sending activations between pipeline stages — it’s more tolerant of slower interconnect between nodes, since communication only happens between stage boundaries, not every layer. **3D parallelism** (tensor + pipeline + data parallelism combined) is what allows training models that don’t fit on a single node at all, combining these two approaches with data parallelism across replicas.

108-110. *(not present in the source document — see note at top)*

111. What is semantic caching for LLM APIs?

Semantic caching stores LLM responses and, for a semantically similar future query, returns the cached result instead of calling the model again — GPTCache is a common implementation that embeds queries and retrieves cached responses by similarity. This dramatically reduces costs for high-traffic applications with a lot of repeated or near-duplicate questions. Mechanically, this is retrieval (Q41-Q42’s embedding-and-similarity-search machinery) applied to your own response history instead of a document corpus. The risk worth flagging: similarity isn’t identity — a cached response for a query judged “similar enough” can be subtly wrong for the actual query asked, if the similarity threshold is set too loosely.

112. What is a model gateway / LLM router?

A model gateway is middleware that intercepts API calls and routes them to the appropriate model or backend, handling authentication, rate limiting, cost tracking, logging, and provider failover in one layer — examples include LiteLLM, Portkey, the AWS Bedrock gateway, and Kong AI Gateway. Provider failover is worth highlighting specifically: it lets you switch from, say, Claude to GPT-4 transparently if one provider has an outage, without changing application code, provided your prompts are portable enough between models (see Q6’s note on prompts not transferring perfectly).

113. What is vLLM and why is it the industry standard?

vLLM is a high-throughput LLM serving library built around PagedAttention (Q102) and continuous batching (Q103), supporting most open-source models and offering an OpenAI-compatible API out of the box — it’s used in production at major AI labs and cloud providers. It’s the reference implementation that made these two techniques broadly accessible: before vLLM, they were research ideas; vLLM turned them into a drop-in serving library, which is a big part of why it became the default rather than teams reimplementing this themselves.

114. What is mixed precision training?

Mixed precision training uses BF16 or FP16 for the forward and backward passes, while keeping FP32 “master weights” for the actual parameter updates — this roughly halves GPU memory usage and speeds up compute without a significant accuracy hit. BF16 is generally preferred over FP16 for training specifically because it has a wider dynamic range and is less prone to overflow. The FP32 master weights detail matters more than it might seem: small gradient updates can underflow to zero in lower-precision formats, so accumulating updates in FP32 and only rounding down to BF16/FP16 for compute avoids silently losing small updates.

115. What is weight tying in language models?

Weight tying shares the input embedding matrix with the output projection (the `lm_head`) weights, forcing the embedding and unembedding spaces to align and significantly reducing parameter count for vocabulary-heavy models. For a model with a 128K vocabulary and a 4096-dimension hidden state, the

embedding matrix alone is roughly 524M parameters — tying input and output embeddings avoids paying for that twice, a meaningful savings specifically for large-vocabulary models.

116. What is the prefill vs decode phase in LLM inference?

Prefill processes the entire input prompt in parallel in one pass — it's GPU compute-bound. **Decode** generates output tokens one at a time sequentially — it's GPU memory-bandwidth-bound. These have genuinely different hardware and optimization needs, which is the mechanical root cause behind the TTFT-vs-TPS distinction in Q105: prefill's parallelism is why TTFT scales gently with prompt length, while decode's sequential, bandwidth-bound nature is why TPS stays roughly constant per model/hardware regardless of how long generation continues.

117. What is dynamic batching in LLM serving?

Dynamic batching groups requests with similar sequence lengths together to minimize padding waste, since padding different-length sequences to a common length wastes compute on the padding tokens themselves; careful variable-length batching and scheduling maximizes GPU utilization as a result. It's distinct from continuous batching (Q103): dynamic batching is about *which* requests get grouped together (by length), while continuous batching is about *when* requests join or leave a batch (mid-flight). Production serving systems typically combine both.

118-120. (not present in the source document — see note at top)

AI AUTOMATION (Q121-140)

121. What is n8n and how is it used with AI?

n8n is an open-source visual automation tool that connects any app or API via nodes, self-hostable for free with 400+ integrations including Claude, OpenAI, and Google. A typical AI workflow follows: trigger → enrich with an LLM → store/notify/act. Self-hosting is the real differentiator versus Zapier (Q122): full control and no per-task pricing, at the cost of running and maintaining

your own infrastructure — worth it when you need complex branching logic Zapier can't express cleanly, or when data residency rules out a SaaS platform.

122. What is Zapier AI and what are Zaps?

Zapier is no-code automation — trigger → action across 6,000+ apps without writing code — and AI Zaps add LLM steps inline for summarizing, classifying, extracting, or translating. It's best suited for non-developers, and more limited than n8n for complex branching logic. Zapier's real advantage is the breadth of pre-built integrations: for straightforward, linear automations across popular SaaS tools, the setup time savings usually outweigh n8n's greater flexibility.

123. What is Make (formerly Integromat)?

Make is a visual automation builder with powerful data mapping and error handling, more flexible than Zapier for complex multi-branch workflows, with strong native AI integrations for OpenAI, Claude, and Gemini modules. It sits between Zapier and n8n on the flexibility spectrum — more branching power than Zapier without n8n's self-hosting overhead, at the cost of a steeper learning curve than Zapier's simpler linear automations.

124. What is LangChain?

LangChain is a Python/JS framework for building LLM applications with chains and agents, providing prompt templates, memory, tools, retrievers, and agent executors, with LCEL (LangChain Expression Language) composing chains declaratively. This repo deliberately does *not* use LangChain — RAG/*.py is hand-rolled Python calling Bedrock, FAISS, and BM25 directly. That's a defensible architectural choice worth articulating: hand-rolled code gives full visibility into exactly what's happening at each retrieval step (useful for debugging the paraphrase-gap issue in Q42), at the cost of writing more glue code yourself than a framework would provide out of the box.

125. What is LlamaIndex?

LlamaIndex is a data framework for connecting LLMs to structured and unstructured data, considered best-in-class specifically for RAG — ingestion, chunking, indexing, and retrieval pipelines — and it supports multi-modal data, knowledge graphs, and agentic retrieval patterns. Its built-in parent-child/

hierarchical chunking abstractions do roughly what RAG/chunking.py implements by hand in this repo — worth knowing both the framework version (for interviews about typical production stacks) and your own hand-rolled version (for explaining the specific design decisions you made).

126. What is DSPy?

DSPy is a framework that compiles high-level task descriptions into optimized prompts, using automatic prompt optimization to replace manual prompt engineering — its “teleprompter” component finds the best few-shot examples and instructions automatically. This is the same concept referenced abstractly in Q33 (APO); DSPy is the concrete implementation of that idea, worth connecting the two explicitly in an interview answer.

127. What is Zapier’s NLA (Natural Language Actions)?

NLA exposes Zapier actions as LLM-callable tools via natural language, letting AI agents trigger any Zapier-connected app through conversational commands — powering AI assistants that can book calendars, send emails, and update CRMs. This is tool use (Q63) applied at the scale of Zapier’s entire app catalog: rather than building custom tool integrations one at a time, NLA exposes thousands of pre-built Zapier actions as agent-callable tools in a single integration.

128-130. *(not present in the source document — see note at top)*

131. What is the OpenAI Assistants API?

The Assistants API provides managed agent infrastructure — threads, messages, tools, file search — handling state management and tool execution automatically. It’s good for rapid prototyping, though it offers less control than building a custom agent loop yourself. The trade-off is the real interview point: managed infrastructure trades control for speed, whereas building your own agent loop gives full visibility and customization at the cost of implementing state management yourself.

132. What is Crew AI?

CrewAI is a multi-agent framework focused on role-based agent collaboration — agents are given roles, goals, and backstories, and collaborate on complex tasks through a simpler API than LangGraph, making it well-suited to content creation and research tasks. “Backstories” sounds like a gimmick but has a real function: persona/role framing (the same underlying mechanic as role prompting, Q27) measurably shapes each individual agent’s output style and focus within the larger multi-agent system.

133. What is a webhook in AI automation?

A webhook is an HTTP endpoint that receives real-time event notifications from other services, triggering automation workflows when events occur — a new email, a form submission — and forms the core building block of real-time AI pipelines: event → enrich → act. Webhooks are the push-based alternative to polling: instead of repeatedly asking “did anything change yet?”, the external service notifies you the instant something happens, which is both faster and cheaper (no wasted polling requests) for event-driven automation.

134. What is Flowise?

Flowise is a visual drag-and-drop builder for LangChain/LlamaIndex-based AI apps, self-hostable, and used to build chatbots, RAG pipelines, and agents visually — it exports to code, making it a good bridge between no-code prototyping and a real production codebase. That export-to-code capability is what distinguishes it from pure no-code tools (Q121-123): you can prototype visually, then get real LangChain/LlamaIndex code out to customize further, rather than being permanently locked into the visual builder.

135. What is Botpress?

Botpress is a conversational AI platform for building production chatbots and agents, combining a visual flow editor, LLM integration, analytics, and multi-channel deployment — used by enterprises to deploy AI assistants across web, WhatsApp, and Slack from one configuration. It’s positioned specifically for conversational/chatbot use cases, versus Flowise’s broader RAG/agent scope; the built-in multi-channel deployment is the concrete feature that saves the most engineering time versus building each channel integration yourself.

136. What is Relevance AI?

Relevance AI is a no-code platform for building AI agents and tools without engineering, connecting to any API, data source, or LLM through a visual builder — popular for sales automation, customer support, and research workflows. It occupies similar territory to n8n/Zapier/Make but is positioned specifically around *agent* building rather than general automation — worth distinguishing tools by primary use case (automation vs. agent-building) rather than treating them as interchangeable.

137. What is a trigger-action paradigm in AI automation?

Every automation starts with a trigger (an event) and executes actions in response; AI adds intelligence to this loop by classifying the trigger, choosing an appropriate action, and personalizing the output — a pattern used across Zapier, n8n, Make, and every major automation platform. Worth stating explicitly: despite different UIs and pricing models, every tool covered above (Zapier, n8n, Make, Flowise, Botpress, Relevance AI) is an implementation of this same trigger → (AI-enhanced) → action loop.

138-140. (not present in the source document — see note at top)

MACHINE LEARNING (Q141-160)

141. What is the bias-variance trade-off?

Bias is error from wrong model assumptions — underfitting. **Variance** is error from the model being too sensitive to the specific training data — overfitting. As model complexity grows, bias falls and variance rises, so there's a sweet spot to find in between. Bagging (like Random Forest) reduces variance through parallel, decorrelated training; boosting (like XGBoost) reduces bias through sequential correction of previous errors. This same underlying trade-off shows up in LLM fine-tuning too: catastrophic forgetting (Q84) is essentially a variance-side failure, where the model overfits to new data and loses its prior generalization.

142. What is gradient descent and its variants?

The core update rule is $\theta \leftarrow \theta - \alpha \cdot \nabla L$ — move the parameters in the direction of the negative gradient, scaled by a learning rate α .

SGD uses a single sample per update; **mini-batch** gradient descent uses a batch of B samples; **Adam** uses adaptive per-parameter learning rates, tracking first and second moment estimates of the gradient for fast convergence with much less manual learning-rate tuning required. Adam (and its variants) is the near-universal default for training and fine-tuning LLMs specifically because that reduced tuning burden matters enormously at scale — a training run costing tens of thousands of dollars in compute is not something you want to re-run repeatedly just to hand-tune a learning rate.

143. What is regularization (L1 vs L2)?

L2 (Ridge) adds a $\lambda ||w||^2$ penalty, shrinking weights proportionally but never all the way to zero. **L1 (Lasso)** adds a $\lambda ||w||$ penalty with constant shrinkage, which does produce genuinely sparse weights (some driven exactly to zero). As a result, L1 induces implicit feature selection, while L2 distributes weight more evenly across all features. “Weight decay” in deep learning (including LLM fine-tuning configs) is effectively L2 regularization applied during optimization — same underlying mechanism, different name.

144. What is cross-validation?

k-fold cross-validation splits the data k ways, trains on $k-1$ folds, validates on the remaining one, then rotates and averages the results — this gives a more reliable performance estimate than a single train/validation split, and stratified CV specifically preserves class balance across folds, which matters for imbalanced datasets (Q154). It’s rarely used in practice for LLM fine-tuning specifically, since training runs are expensive enough that a single held-out validation/test split (as this repo’s RAG eval does — an 11-question held-out set) is the norm rather than paying for k separate training runs.

145. What is XGBoost and why is it powerful?

XGBoost is gradient boosting using a second-order Taylor expansion of the loss function (leveraging the Hessian, not just the gradient), with L1/L2 regularization applied directly on tree structure to prevent overfitting, plus sparse-aware split finding and cache-aware computation that make it extremely fast in

practice. It remains the dominant choice for tabular/structured-data problems even in the current LLM era — worth remembering that “AI engineer” doesn’t mean “only LLMs”: classical gradient boosting is still the right tool for many real business problems (fraud detection, churn prediction) that don’t involve text generation at all.

146. What is the ROC-AUC metric?

ROC plots the true positive rate against the false positive rate across all classification thresholds, and AUC is the area under that curve — a threshold-independent performance measure, where $AUC=1$ is a perfect classifier and $AUC=0.5$ is no better than random. On imbalanced data, PR-AUC is generally preferred, since it focuses on performance for the minority class specifically. Your RAG eval doesn’t use either of these — it uses hit rate and MRR (Q59-adjacent), because retrieval is a *ranking* problem (“is the right document near the top?”) rather than a binary classification problem, which is what ROC-AUC and PR-AUC are actually designed to measure.

147. What is SHAP and how does it explain model predictions?

SHAP assigns each feature a Shapley value representing its average marginal contribution to a prediction, satisfying the formal properties of efficiency, symmetry, the dummy axiom, and additivity; TreeSHAP computes exact Shapley values for tree-based models in polynomial time, making it practical to use even on large models. This is for classical ML model interpretability — explaining why an XGBoost model predicted a given outcome; it doesn’t directly apply to LLM output explainability, which remains a much less solved problem, so it’s worth not conflating the two very different “explainability” needs in an interview.

148-150. (not present in the source document — see note at top)

151. What is PCA (Principal Component Analysis)?

PCA is linear dimensionality reduction that finds the axes of maximum variance in the data, computed via singular value decomposition (SVD) of the data matrix, reducing dimensions while preserving as much of the original variance as possible — commonly used for visualization, noise reduction, and

preprocessing before other ML steps. It's rarely applied to LLM embeddings in production RAG systems, unlike classical ML feature vectors — reducing embedding dimensionality with PCA generally loses too much retrieval quality; if you need smaller vectors, choosing a smaller embedding model outright (say, 768-dim instead of 3072-dim) usually beats PCA-reducing a larger model's output.

152. What is the curse of dimensionality?

As the number of features grows, data becomes increasingly sparse in the resulting high-dimensional space — the volume grows exponentially with dimension — and distance metrics lose their discriminative meaning as a result, causing nearest-neighbor search quality to degrade rapidly. Mitigations include PCA, feature selection, regularization, or learned representations that compress the meaningful signal into fewer dimensions. This is directly relevant to why embedding dimension (Q8/Q55) is a real design choice rather than an arbitrary one: very high-dimensional embeddings can suffer this exact degradation in nearest-neighbor retrieval quality, part of why 768-1536 dimensions is a common sweet spot rather than always maximizing dimension count.

153. What is precision vs recall?

Precision = $TP / (TP + FP)$ — of everything predicted positive, how much actually was. **Recall** = $TP / (TP + FN)$ — of everything that was actually positive, how much did the model correctly identify. Prefer optimizing for precision when false positives are costly (spam filtering), and recall when false negatives are costly (cancer screening, where missing a true case is far worse than a false alarm). These formulas apply directly to RAG retrieval evaluation too, just renamed: Precision@k (of the top-k retrieved chunks, how many are actually relevant) and Recall@k (of all relevant chunks in the corpus, how many made it into the top-k) are the same math applied to retrieved documents instead of classification predictions.

154. What is class imbalance and how is it handled?

When one class has far more samples than another, models naturally become biased toward predicting the majority class. Standard fixes include oversampling the minority class (SMOTE), undersampling the majority class, applying class weights directly in the loss function, or evaluating with PR-AUC (Q146) instead of plain accuracy or ROC-AUC — and ensemble methods (Q157) are

generally more robust to imbalance than single models. A subtler version of this same problem shows up in fine-tuning data curation (Q95): if an instruction dataset has 1,000 examples of one task type and only 10 of another, the fine-tuned model ends up biased toward the overrepresented task — same underlying problem, same fix (balance the dataset before training).

155. What is early stopping?

Early stopping monitors validation loss during training and halts once it stops improving for a set number of epochs (patience), acting as an implicit form of regularization by limiting the model's effective capacity to overfit — the checkpoint saved should be the one with the best validation loss, not simply the one at the very end of training. This is the practical mechanism behind the learning-rate diagnostic in Q96: “if training loss increases after the first epoch, the LR is too high” is exactly the kind of signal early stopping is designed to catch automatically, so you don't have to manually watch every single training run.

156. What is a confusion matrix?

A confusion matrix is a grid of actual versus predicted class counts — rows are actual classes, columns are predicted classes — where the diagonal represents correct predictions and off-diagonal cells represent errors, broken down by type. From it you can derive precision, recall, F1, and specificity for a complete error analysis. For RAG's per-question evaluation, the analogous breakdown is looking at hits versus misses *by query difficulty* (exact vs. paraphrase, as this repo's eval set already does) rather than a full confusion matrix — the same spirit of “don't just look at the aggregate number,” adapted from classification to a ranking problem.

157. What is ensemble learning?

Ensemble learning combines multiple models whose individual errors are largely uncorrelated, so averaging their predictions cancels out much of the noise. **Bagging** (as in Random Forest) trains models in parallel on different data subsets to reduce variance; **boosting** (as in XGBoost) trains models sequentially, with each new model focusing on correcting the previous ones' errors, reducing bias. Your RAG/hybrid_search.py is conceptually an ensemble too: BM25 and dense retrieval are two different retrieval “models” whose errors are largely uncorrelated (a paraphrase query dense retrieval might be one BM25

misses, and vice versa), and RRF fusion (Q42) is the combination step — the same underlying principle as bagging or boosting, applied to retrieval instead of classification.

158-160. (not present in the source document — see note at top)

DEEP LEARNING (Q161-180)

161. What is backpropagation?

Backpropagation applies the chain rule to compute the gradient of the loss with respect to every parameter in the network, with gradients flowing backward from the output layer to the input layer using activations cached during the forward pass — this automatic differentiation process is the foundation of training essentially all neural networks. Every fine-tuning method covered above (Q81-Q97) relies on this unchanged; what differs between full fine-tuning, LoRA, and QLoRA is *which* parameters actually receive gradient updates, not the underlying gradient computation mechanism itself.

162. What is residual connection (ResNet)?

A residual (skip) connection computes $\text{output} = F(x) + x$, letting the gradient flow through the skip path directly during backpropagation — this is what enables training networks 100+ layers deep by alleviating vanishing gradients, and it also lets the network learn small corrective residuals rather than having to learn a full mapping from scratch at every layer. This exact mechanism is reused, unmodified, in every Transformer block (Q175) — the “Add” step in “Add & LayerNorm,” applied after both self-attention and the feed-forward layer, is this same residual connection. Without it, a 32+ layer LLM simply wouldn’t train — this ResNet-era idea is load-bearing infrastructure for every modern LLM.

163. What is layer normalization?

Layer normalization normalizes activations across the feature dimension *per sample*, rather than across a batch — making it batch-size independent, which is why it’s preferred over BatchNorm for sequence models and Transformers specifically: sequences in a batch have varying lengths (unlike fixed-size

images), which makes computing meaningful batch statistics awkward, and per-sample normalization sidesteps that problem entirely. RMSNorm (used in LLaMA) removes the mean-centering step of standard LayerNorm, running faster with similar performance.

164. What is the Vision Transformer (ViT)?

ViT divides an image into fixed-size patches, embeds each patch as a token, prepends a class token, and adds positional embeddings to the patch tokens — then processes the whole sequence through a standard Transformer. It's competitive with CNNs at scale, with DINOv2 demonstrating excellent transfer learning results. The elegant insight worth stating: ViT doesn't invent new machinery for images at all — it just reframes an image as a sequence of "tokens" (patches) and runs it through the exact same Transformer architecture (Q175) used for text. Positional embeddings matter more here than in text, since patch order isn't inherently sequential the way word order is.

165. What is self-supervised learning (SSL)?

SSL trains representations without human-provided labels, using structure in the data itself as the supervisory signal. **Contrastive** methods (SimCLR, MoCo) contrast augmented views of the same image against different images. **Masked** methods (MAE) mask out patches and train the model to reconstruct the missing pixels, learning rich features in the process. LLM pretraining (Q86) is itself a form of self-supervised learning: next-token prediction uses the text itself as the label (the actual next word), requiring no human annotation at all — which is exactly why pretraining can scale to trillions of tokens.

166. What is knowledge distillation in deep learning?

A small "student" model is trained to match a large "teacher" model's soft probability outputs, rather than just hard ground-truth labels — soft targets at higher temperature (Q3) reveal inter-class similarity information a hard label alone discards. DistilBERT is the canonical example: 40% smaller than BERT, 60% faster, while retaining 97% of its performance. The temperature parameter used to soften the teacher's distribution here is the same one that controls generation randomness in Q3 — same mechanism, different application.

167. What is GELU activation and why is it used in transformers?

GELU is defined as $\text{GELU}(x) = x \cdot \Phi(x)$ (where Φ is the standard Gaussian CDF), producing a smooth, non-zero gradient everywhere — unlike ReLU, which has a hard zero for all negative inputs. That slight negative output for small negative x gives better gradient flow during training, and GELU is used in BERT, GPT, and virtually all modern Transformer architectures as a result. This matters specifically because ReLU's hard zero can cause “dead neurons” that never receive a gradient and stop updating entirely; GELU's smoothness avoids that failure mode, which is a meaningful part of why it became the default over ReLU inside Transformer feed-forward layers (Q175).

168-170. *(not present in the source document — see note at top)*

171. What is contrastive learning?

Contrastive learning trains an encoder to embed similar pairs close together in vector space and dissimilar pairs far apart. SimCLR, for instance, trains on two differently-augmented views of the same image, which should end up closer together than views of different images; the NT-Xent loss is commonly used, and a large batch size matters critically here, since it provides many negative examples per positive pair — this is exactly how embedding models for RAG (Q8/Q55) are trained in the first place: models like Titan v2 or BGE-M3 use a contrastive objective so that semantically related text ends up close together in vector space, which is the entire mechanism RAG/embed.py and faiss_search.py depend on to work at all.

172. What is the encoder-decoder architecture?

The encoder uses bidirectional attention to read and represent the full source sequence at once; the decoder uses causal (masked) self-attention combined with cross-attention to the encoder's output, generating the target sequence one token at a time while attending back to the relevant parts of the source. This is the third architectural variant alongside encoder-only (BERT-style, used by your reranker's CrossEncoder) and decoder-only (GPT/Claude-style, used by your generation LLM) — see Q175 for the full three-way comparison. Encoder-decoder architectures (T5-style) are less common for modern general-purpose chat LLMs but remain standard for translation and some summarization systems.

173. What is dropout and how does it prevent overfitting?

Dropout randomly zeros out neuron activations with probability p during training, forcing the network to build redundant representations since no single neuron can rely on any particular other neuron being present. At inference time, all neurons are active but outputs are scaled by $(1-p)$ to compensate. The “ensemble effect” is worth expanding on: because dropout samples a different sub-network on every forward pass during training, it effectively approximates training an ensemble of many smaller networks (connecting back to Q157’s ensemble learning) — implemented implicitly within a single model, rather than by explicitly training separate ones.

174. What is weight initialization and why does it matter?

Poor initialization causes vanishing or exploding gradients right from the start of training, before the model has even had a chance to learn anything. **Xavier initialization** sets $\text{Var}(w) = 2 / (\text{fan_in} + \text{fan_out})$, well-suited to tanh/sigmoid activations; **He initialization** sets $\text{Var}(w) = 2 / \text{fan_in}$, specifically optimized for ReLU, and is the default in most modern deep learning frameworks. Worth connecting to Q167: since GELU, not ReLU, is the standard Transformer activation, modern LLM implementations often use initialization schemes closer to Xavier, or specially-tuned variants, rather than plain He initialization, which was derived specifically for ReLU’s particular characteristics.

175. What is the Transformer architecture overview?

See Q13 for the complete architecture and how attention fits into the Transformer block. Three architectural variants matter most for interviews: **encoder-only** (bidirectional attention, e.g. BERT or your reranker’s CrossEncoder — can see the whole input at once), **decoder-only** (causal/masked attention, e.g. GPT/Claude — each token only sees itself and prior tokens, which is what makes autoregressive generation from Q16 possible), and **encoder-decoder** (both, e.g. T5 or BART — encoder processes the whole input bidirectionally, then decoder attends to the encoder’s output while generating one token at a time). The differences matter: BERT’s bidirectional context is why it’s great for classification and embedding, while GPT’s causal masking is why it’s great for generation. For a production LLM you’re using (Claude, GPT-4), the decoder-only architecture is standard — it’s the only one that

naturally does autoregressive text generation. The $O(n^2)$ attention complexity is why long context is expensive (Q7) and why optimizations like PagedAttention (Q102) exist.

176. What is a VAE (Variational Autoencoder)?

A VAE's encoder outputs the mean and variance of a Gaussian posterior distribution $q(z|x)$ over the latent space, rather than a single deterministic point. The **reparameterization trick** — $z = \mu + \sigma \cdot \epsilon$, where ϵ is sampled independently — makes this differentiable: sampling itself isn't a differentiable operation, so you can't backpropagate through it directly, but rewriting the sample as a deterministic function of μ and σ (with the randomness isolated in ϵ , which needs no gradient) makes the whole pipeline trainable end-to-end. The loss combines reconstruction error with KL divergence, which forces the latent space into a structured, well-behaved form rather than an arbitrary one.

177-180. (not present in the source document — see note at top)

CAREER & PRODUCTS (Q181-200)

181. How do you evaluate an LLM for a production use case?

Start by defining task-specific metrics — accuracy, faithfulness, latency, refusal rate — then layer evaluations from cheap and automated to expensive and precise: automated benchmarks first, then LLM-as-judge, then human gold-standard review for the highest-stakes cases. Gate deployment on this eval suite in CI/CD, running it on every model or prompt change before it ships. Your RAG/eval/ directory is a concrete, working example of exactly this pattern applied to retrieval: `evaluate_retrieval.py` is an automated benchmark you can (and should) gate prompt, model, or chunking changes against before deploying, rather than eyeballing a handful of example queries and hoping quality didn't regress.

182. What is red-teaming for AI models?

Red-teaming is adversarial testing that deliberately tries to make a model produce harmful outputs, covering jailbreaks, prompt injection, bias elicitation, and privacy leakage — both Anthropic

and OpenAI combine external red teams with automated red-teaming tools for this. It connects directly to Q24: red-teaming is the *process* of systematically testing for the exact vulnerabilities Q24 describes, rather than simply hoping your mitigations work. Treat it as a continuous practice — run it before every major prompt or model change — rather than a one-time pre-launch checkbox.

183. What is an AI product tech stack for 2025?

A typical 2025 stack: LLM layer (Claude or GPT-4o), orchestration (LangGraph or LlamaIndex), memory (Postgres+pgvector or Pinecone), evaluation (RAGAS, LangSmith), and observability (Langfuse, Helicone, Braintrust). For serving specifically, it's worth separating three genuinely different categories that get blurred together: **vLLM/TGI** are the actual LLM serving *libraries* (Q113); **AWS Bedrock** is a managed LLM API with no serving infrastructure to run yourself; **Modal** is a serverless *compute platform* that can *host* something like vLLM, but isn't itself a serving library — a distinction worth getting right if asked to justify a stack choice in an interview.

184. What is responsible AI and why does it matter?

Responsible AI ensures systems are safe, fair, transparent, and accountable, covering bias, privacy, security, environmental impact, and human oversight — and regulatory frameworks like the EU AI Act (Q186) increasingly mandate this rather than leaving it purely voluntary. Several techniques covered elsewhere already map onto these pillars in practice: RAG's citation and faithfulness enforcement (Q56) supports transparency, red-teaming (Q182) supports safety, and access-control-aware metadata filtering (Q53) supports privacy — responsible AI isn't a separate add-on practice so much as a lens on techniques you'd likely build anyway for quality reasons.

185. What is AI observability?

AI observability monitors LLM inputs, outputs, latency, cost, and errors in production, using tools like Langfuse, Helicone, Braintrust, W&B Weave, or LangSmith — essential for debugging hallucinations and catching prompt regressions before they reach users at scale. Your repo's timestamped `eval/results/*.json` files are a lightweight, homegrown version of this same idea: a record of retrieval quality over time you could diff to catch regressions,

the same underlying goal as a dedicated observability tool, just without the dashboard and alerting layer a tool like Langfuse would add on top.

186. What is the EU AI Act and how does it affect AI engineers?

The EU AI Act is risk-based regulation with four tiers — unacceptable, high, limited, and minimal risk — where high-risk applications (hiring, credit, healthcare) require documentation and auditing, and General Purpose AI models like GPT-4 or Claude face transparency requirements of their own. The practical engineering implication of “documentation and auditing”: if you’re building a high-risk application, your eval harness (Q181/Q77) and observability logs (Q185) stop being purely quality tools and become compliance artifacts you may need to produce on request.

187. What is AI for code review and how does it work?

An LLM analyzes pull request diffs to find bugs, style violations, and security issues, with tools like GitHub Copilot, CodeRabbit, and Graphite Automations integrating directly into CI/CD — best used for catching obvious issues, while humans remain necessary for architectural judgment. That caveat is worth taking seriously and connects to Q19’s honest assessment of LLM limitations: code review tools are good at pattern-matching against known bug and style categories, but weaker at judging whether a design decision is architecturally sound for a system’s specific future needs.

188-190. *(not present in the source document — see note at top)*

191. What is an AI side hustle and what’s realistic?

Realistic paths include AI freelancing (automation, chatbots, RAG systems on Upwork or Fiverr) and AI-powered content (newsletters, YouTube tutorials, prompt packs on Gumroad) — but results depend entirely on skill, consistency, and how specific a niche you occupy, not on the tools themselves. Notably, the RAG system built in this repo — with real measured eval numbers (90.9% hit rate) rather than just “I read about RAG” — is exactly

the kind of concrete, demonstrable work that differentiates a freelance or portfolio pitch from someone who's only done tutorials.

192. What is the AI engineering career path?

A typical progression: entry-level as a prompt engineer or AI assistant developer building on top of existing APIs; mid-level as an AI engineer building RAG systems, fine-tuning pipelines, agent pipelines, and evals; senior as an ML engineer or AI researcher working on training, architecture, and infrastructure. By this framework, the RAG work in this repo — hybrid search, HyDE, parent-child retrieval, real eval numbers — is solidly mid-level “AI engineer” territory already; the fine-tuning pipeline (Q81’s suggested next step) is the natural next concrete project to round out that profile before senior-level training and architecture work becomes relevant.

193. What skills does an AI engineer need in 2025?

Core skills: prompt engineering, Python, and LLM APIs (Claude, OpenAI, Gemini); RAG pipelines, vector databases, and agent frameworks (LangGraph, LlamaIndex); and ML fundamentals, fine-tuning (especially LoRA), evaluation, and production deployment. Checking this repo against the list: RAG pipelines , vector databases  (FAISS), evaluation  (real metrics in eval/), Python and LLM APIs  (Bedrock). Fine-tuning (LoRA) and agent frameworks are the two items not yet represented — see Q81 and Q54 for concrete starting points on each.

194. What is AI for interview preparation?

Use an LLM to roleplay technical and behavioral interviews with feedback, generate domain-specific question banks, and practice explaining concepts simply; you can also research target companies, industries, and role requirements with AI assistance. This document is a direct instance of the “domain-specific question bank” idea — now with references to your own real implementation folded in, so your answers aren’t just recited definitions but backed by “here’s the code and the numbers,” which is a materially stronger interview answer than a memorized textbook definition.

195. What is a Claude Project and how do AI creators use it?

A Claude Project is a persistent workspace with uploaded docs and custom system instructions, letting creators upload brand guidelines, past content, and audience persona so the AI stays aligned across sessions — eliminating repeated context-setting and enabling consistent output at scale. This exact project (“RAG & Fine Tuning”) is a working example: the learning plan, best-practices checklist, and this enhanced interview guide all persist here across sessions, so context doesn’t need to be re-established from scratch in every conversation.

196. What is AI content automation for creators?

Long-form content gets repurposed through a pipeline — video → transcript → blog → tweets → carousel — using tools like Descript for video, Castmagic for podcasts, and Claude for text transformation, where AI handles the mechanical production work while humans retain strategy, voice, and authentic connection. That division of labor — AI executes well-scoped, repeatable sub-tasks, humans retain judgment over goals and priorities — is a useful general heuristic beyond content creation specifically; it’s the same principle behind where RAG (Q11) and agents (Q61) fit into a broader system.

197. What is multimodal AI and its applications?

Multimodal AI processes text, images, audio, and video together within one model, with applications spanning document analysis, image captioning, and video understanding — GPT-4o, Claude 3.5, and Gemini 1.5 Pro are current frontier multimodal models. This is relevant to this repo’s future direction if the notes corpus ever includes diagrams or screenshots: a multimodal embedding and retrieval approach would be needed to make image content in markdown notes searchable, beyond the current text-only chunking.py/embed.py pipeline.

198-200. *(not present in the source document — see note at top)*

Summary

Every original question is answered above as a single, complete, correct answer — nothing split into separate “original” and “enhanced” blocks, nothing dropped, nothing renumbered from the source document. Diagrams and code examples are included where they genuinely clarify the mechanism (the Transformer block in Q175, the RAG pipeline in Q41, the attention formula in Q13, the tokenization example in Q2), not decoratively on every entry.

The 📌 callouts — concentrated in the RAG section (Q41–Q57) with a few cross-references elsewhere — point at real code and real measured numbers from this repo’s RAG/ folder. Use those as your strongest interview talking points: “I built this and measured 90.9% hit rate” beats reciting a textbook definition every time.

Fine-tuning (Q81-100) has no implementation in this repo yet — answered from first principles, with Q81 and Q95 marking the concrete starting points for building a real pipeline there, the same way RAG/ already exists for retrieval.

Last updated: September 18, 2026